



RoboCore — System Architecture

Read this first. This document is the single source of truth for the RoboCore project. It is written so that a new contributor (human or AI) can read *only this file* and understand the robot, the code layout, the data flow, the protocols, and how to build/run everything. The `firmware/` tree (the ESP32 project + the VESC LispBM scripts) and the `esp32_bridge` node are implemented; most of the `software/` ROS 2 packages are still scaffolding. This document defines the design and tracks what is built vs. planned.

Table of contents

1. [What this project is](#)
 2. [Lineage and what changed from the legacy repo](#)
 3. [System overview & data flow](#)
 4. [Hardware inventory](#)
 5. [Compute domains: what runs where](#)
 6. [Repository layout](#)
 7. [Firmware \(ESP32\)](#)
 8. [CAN bus & motor protocols](#)
 9. [ESP32 \$\rightleftarrows\$ Jetson binary UART protocol](#)
 10. [Jetson software \(relay\)](#)
 11. [Workstation software \(operator\)](#)
 12. [ROS 2 topic reference](#)
 13. [Operating modes, RC link & control loops](#)
 14. [Coordinate frames / TF](#)
 15. [Networking](#)
 16. [Safety & e-stop](#)
 17. [Build & run](#)
 18. [Open decisions & TODO](#)
 19. [Glossary](#)
-

1. What this project is

RoboCorea is the software + firmware monorepo for a **tracked search-and-rescue robot** built for the **RoboCup Rescue** category. The robot is a differential (tank) drive platform with four articulated flippers for climbing obstacles, a 6-DOF manipulator arm, and a sensor suite for victim/hazard detection. It is **teleoperated** by a human operator at a remote workstation; autonomous navigation is planned but deferred (see §18).

The system spans three compute domains that must cooperate:

- **ESP32 PCBs** (two identical custom PCBs) — hard-real-time I/O split by firmware role: chassis/base on one board, arm on the other.
- **NVIDIA Jetson Orin Nano** (on the robot) — a ROS 2 relay between the robot and the operator, plus on-robot sensors.
- **Operator workstation** (a laptop) — the GUI, computer vision, and the arm control stack. The human also holds the RC transmitter here.

2. Lineage and what changed from the legacy repo

This project supersedes an earlier monorepo, **TMR2026_Rescue**, kept under `reference/TMR2026_Rescue/` as inspiration **only — not a source of truth**. That repo was largely "vibe coded": the main functions work, but expect logic bugs. It also contained code for *two* robots — "**Dicerox**" (the predecessor of this robot) and "**Jaguar**" (an unrelated arm platform, **ignore it**).

The legacy repo's **most current firmware lives on its firmware branch** (a refactored `shared/firmware/` with a clean `lib/` module structure). The checked-out working tree may be on `main`, which is older. When mining the legacy code, prefer `git show origin/firmware:<path>`.

RoboCorea keeps the good ideas (the binary UART protocol, the RC mode/e-stop

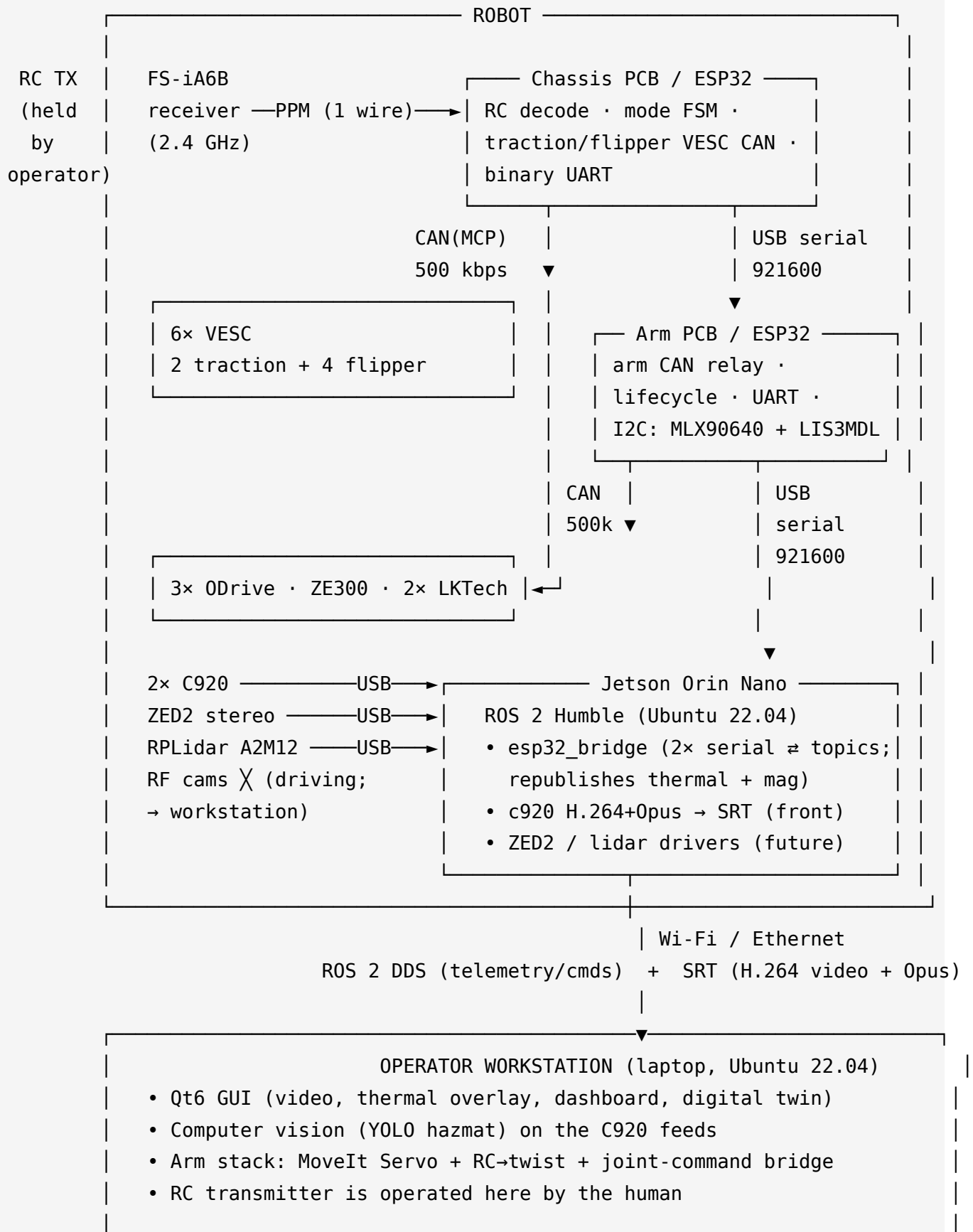
state machine, the flipper position-PID concept, the Qt GUI, the MoveIt arm stack) and makes these deliberate changes:

Area	Legacy (Dicerox)	RoboCorea (this project)
Repo scope	Two robots (Dicerox + Jaguar), many scattered workspaces	One robot, one firmware project + one ROS 2 workspace
CAN transceiver	MCP2515 over SPI	On-board SMD MCP2515 over SPI (permanent); TWAI + SN65HVD230 kept compilable as a future option
Traction drivers	ODrive (CANSimple)	VESC (all 6 base motors are identical VESCs)
Flipper drivers	VESC	VESC; position loop runs on the VESC (LispBM), not the ESP32
Base position feedback	Quadrature encoders on ESP32 PCNT	Hall feedback via the VESC over CAN (no separate encoders); the VESC tachometer is forwarded for wheel odometry
Odometry	Quadrature encoders	ZED2 VIO + VESC-tachometer wheel odom , to be fused by a planned <code>robot_localization</code> EKF
Thermal camera	MLX90640 on the ESP32 I2C bus	MLX90640 on the arm-PCB ESP32 I2C bus, relayed by <code>esp32_bridge</code> (Jetson I2C proved unreliable)
Magnetometer	QMC5883L / LIS3MDL on the ESP32 I2C bus	LIS3MDL on the arm-PCB ESP32 I2C bus, relayed by <code>esp32_bridge</code>

Area	Legacy (Dicerox)	RoboCorea (this project)
Sensors on ESP32	BNO055 IMU, QMC5883L mag, MQ2 gas	Thermal + mag on the arm PCB (read by the arm ESP32, forwarded over UART). IMU dropped — orientation now from the ZED2 ; gas sensor dropped
Robot↔PC link	micro-ROS <i>or</i> serial	Plain serial + a ROS 2 bridge node (preferred)
Cameras	Streamed from robot over the network (GStreamer)	RF analog cams → USB digitizers at the workstation (driving) + 2× C920 on the Jetson → onboard H.264 (+Opus) over SRT (inspection/CV); thermal still via ROS (now from the arm PCB through the bridge)
Arm	ODrive J1–J3 + ZE300 J4 + LKTech J5–J6	Unchanged — same mixed-CAN arm

Everything else (the arm CAN protocols, the binary protocol framing, the GUI panels) carries over, adapted to the single-robot reality.

3. System overview & data flow



RF cameras —5.8 GHz→ USB digitizer → /dev/videoN (webcams)

Two independent links reach the robot:

1. **The RC link** (2.4 GHz, FlySky): operator's handheld transmitter → FS-iA6B receiver on the robot → PPM → ESP32. This is the low-latency teleop path and works even if the network/ROS link is down (failsafe to neutral).
2. **The network link** (Wi-Fi/Ethernet, ROS 2 DDS): Jetson ⇔ workstation. Carries telemetry, sensor data, PPM calibration, and **arm joint commands computed on the workstation** (from its gamepad, not the RC sticks).

Video uses two paths, neither of them DDS. The two **RF analog (FPV) cameras** are the *driving* feed: they transmit over their own 5.8 GHz RF link to receivers + USB digitizers at the workstation, where they appear as ordinary /dev/videoN webcams (they never touch the robot's computers). The two **C920 USB cameras on the Jetson** are the *inspection / CV* feed: the Jetson streams their onboard H.264 (and the front camera's mic as Opus) over **SRT** straight into the GUI — a dedicated low-latency media transport tuned for the degraded competition link, **not** ROS/DDS. Only the tiny **thermal** image traverses ROS (read on the **arm PCB**, forwarded over UART, and republished as an Image topic by esp32_bridge). Rationale + tuning: software/ros2_ws/src/gui/README.md .

4. Hardware inventory

Component	Qty	Connection	Notes
6S 8 Ah LiPo battery	1	—	~25.2 V full, ~22.2 V nominal. Powers everything.
Custom PCB w/ ESP32	2	USB serial → Jetson; CAN per board	Identical hardware and firmware. ROBOCOREA_BOARD_ROLE selects chassis or arm at build time.

Component	Qty	Connection	Notes
BLDC motor (traction)	2	VESC → CAN	Left/right tracks, differential drive. 23.333:1 reduction. Hall sensors.
BLDC motor (flipper)	4	VESC → CAN	FL, FR, RL, RR. Position-controlled (0–360°, wraps). 100:1 reduction. Hall sensors.
VESC mini 6.7 Pro	6	CAN (MCP2515)	One per base motor. Traction = velocity (<code>SET_RPM</code>); flippers = position via an on-board LispBM script. Set each to 500 kbps CAN before bus-up.
FlySky FS-iA6B receiver	1	PPM → chassis ESP32 GPIO4	6 channels multiplexed on one PPM wire. Paired with the operator's FlySky transmitter.
6-DOF robotic arm	1	Mixed CAN → arm ESP32	J1–J3 ODrive, J4 ZE300, J5–J6 LKTech. See §8. Driven on the arm PCB's separate CAN bus .
NVIDIA Jetson Orin Nano	1	USB to both ESP32 PCBs; USB for sensors	Runs ROS 2 Humble on Ubuntu 22.04. Robot-side relay.
MLX90640 thermal camera	1	I2C → arm-PCB ESP32	32×24 thermal array. Read by the arm ESP32 (low-priority task, ~4 Hz), quantised + sent over UART; <code>esp32_bridge</code> republishes <code>/sensors/thermal</code> .
LIS3MDL magnetometer	1	I2C → arm-PCB ESP32	Heading reference. Read by the arm ESP32 (50 Hz), sent over UART; <code>esp32_bridge</code> republishes <code>/sensors/mag</code> .
ZED2 stereo camera	1	USB → Jetson	Visual-inertial odometry and the robot's orientation/IMU source (replaces the dropped BNO055). Feeds the planned odometry EKF +

Component	Qty	Connection	Notes
			future SLAM/nav.
RPLidar A2M12	1	USB → Jetson	2D lidar for future SLAM/nav.
RF (FPV) camera	2	5.8 GHz RF → USB digitizer → workstation	Appear as <code>/dev/videoN</code> webcams. Driving feed (low-latency, low-res).
Logitech C920 Pro	2	USB → Jetson	Onboard H.264 (UVC) + built-in mic. Inspection / CV feed — streamed to the GUI as H.264 (front cam also Opus audio) over SRT (§11).

Buses summary

- **Chassis ESP32 I2C**: unused in the normal robot build; the passive sensors live on the *arm* PCB.
- **Arm ESP32 I2C** (`SDA=GPI021` , `SCL=GPI022` , 400 kHz): **MLX90640** thermal camera (0x33) + **LIS3MDL** magnetometer. Read by the arm ESP32 in low-priority tasks that never touch the CAN bus.
- **Chassis ESP32 CAN** via an **on-board SMD MCP2515** over SPI (`CS=GPI05` , `SCK=GPI018` , `MISO=GPI019` , `MOSI=GPI023` , 8 MHz crystal), **500 kbps**: the 6 VESCs only (2 traction + 4 flippers).
- **Arm ESP32 CAN** via the same MCP2515 hardware, **500 kbps**: 3 ODrives + 1 ZE300 + 2 LKTech. The two PCBs do **not** share one CAN bus.
- **ESP32 PCBs ⇄ Jetson**: two USB serial links, **921600 baud**, binary framed protocol with board identity (§9).
- **Jetson I2C (GPIO)**: unused — the thermal camera + magnetometer moved to the arm PCB.
- **Jetson USB**: ZED2, RPLidar A2M12, **2× Logitech C920** (H.264 + Opus → SRT).
- **Workstation USB**: 2× RF-camera digitizers.

Pin numbers and CAN IDs above are taken from the working `1.ino` / `2.ino` firmware. Re-confirm against the custom PCB before flashing.

5. Compute domains: what runs where

Process / responsibility	Chassis ESP32	Arm ESP32	Jetson	Workstation
RC/PPM decode, mode FSM, e-stop	✓			
Traction differential mixing → VESC RPM	✓			
Flipper stick → target angle (loop closed on the VESC)	✓			
Arm CAN relay (ODrive/ZE300/LKTech)		✓		
LIS3MDL magnetometer read (I2C)		✓		
MLX90640 thermal read + quantise (I2C, low-prio)		✓		
Binary UART telemetry/commands	✓	✓		
Serial ⇌ ROS 2 bridge (esp32_bridge)			✓	
Thermal/mag UART → /sensors/thermal + /sensors/mag			✓	
C920 H.264 + Opus → SRT (GStreamer streamer)			✓	
ZED2 + RPLidar drivers (future SLAM)			✓	
Operator GUI (Qt6)				✓
C920 A/V SRT receive + speech (Vosk)				✓
Computer vision (YOLO hazmat) on the C920 streams				✓

Process / responsibility	Chassis ESP32	Arm ESP32	Jetson	Workstation
Movelt Servo + arm IK				✓
Gamepad → arm twist mapping (joystick_servo)				✓
Holds the RC transmitter				👤

Why the arm runs on the workstation: the Jetson is intentionally a thin relay. The arm joint commands are computed on the workstation from the gamepad/servo stack, then sent **back** down: workstation → (network) → Jetson `esp32_bridge` → (USB serial) → arm ESP32 → (arm CAN) → arm drivers. See §13.

6. Repository layout

```
RoboCorea/
├── firmware/
│   ├── ESP/                # ESP32 PlatformIO project – IMPLEMENTED (see §7)
│   └── VESC/               # LispBM scripts that run ON the VESCs
│                           # flipper_position.lisp – flipper position loop (§7.3)
│
├── software/
│   ├── ros2_ws/            # ONE ROS 2 Humble workspace, shared by Jetson
│   │   └── src/
│   │       ├── esp32_bridge/ # Serial ⇄ ROS 2 bridge – IMPLEMENTED
│   │       ├── dicerox_mapping/ # 2D SLAM (slam_toolbox) – IMPLEMENTED (§10)
│   │       ├── rescue_nav/    # Nav2 + EKF + sim + /cmd_vel path – IMPLEMENTED (§10)
│   │       └── gui/           # Qt6 operator console – video/dashboard/odometry/
│   │                           # speech/twin/CV-filters/config-dialogs IMPLEMENTED
│   │                           # (other packages below are planned)
│   └── sim_ws/             # SEPARATE Gazebo (gz-sim/Ignition) sim workspace
│                           # (tracked_robot_*) – standalone prototype, NOT wired
│                           # into ros2_ws; rescue_nav has its own Gazebo-Classic sim
│
└── reference/
    ├── architecture.md     # ← THIS FILE. The source of truth.
    ├── draft.txt           # Original hand-written project outline.
    └── TMR2026_Rescue/     # Legacy code. Inspiration only, NOT truth.
```

A **single** `ros2_ws` is used on both Linux machines because they share custom message types and many launch files; hosts simply run different subsets.

Implemented so far: the ESP32 firmware (`firmware/ESP`), the `esp32_bridge` node, and the `gui` package's video + dashboard + odometry + speech + digital twin + in-GUI CV filter + config-dialog subsystems. The rest of the packages below are planned:

Package	Lang	Runs on	Purpose
<code>rescue_interfaces</code>	msg/srv	both	Shared custom message & service definitions. (arm saved-pose services:

Package	Lang	Runs on	Purpose
			SavePose/GoToPose/DeletePose/ListPoses; named-map services: SaveMap/ListMaps/LoadMap/DeleteMap)
esp32_bridge	Python	Jetson	Serial binary protocol $\rightleftharpoons$ ROS 2 topics. The relay core; also decodes the arm-PCB thermal/mag UART frames $\rightarrow$ /sensors/thermal (Image) + /sensors/mag (MagneticField) and relays /sensors/enable_mask down. (implemented)
mlx90640_msgs	msg	both	ThermalStatus diagnostics msg published alongside /sensors/thermal . (implemented)
dicerox_mapping	mixed	Jetson	2D SLAM: slam_toolbox + ZED2 planar odom (zed_planar_odom) + RPLidar scan reframer (scan_frame_republisher). Map save + localization. (implemented)
rescue_nav	Python	Jetson/WS	Navigation layer on top of dicerox_mapping : Nav2 (planner + RegulatedPurePursuit + costmaps + waypoint follower), robot_localization EKF with adaptive covariance, a Gazebo <i>Classic</i> sim, the waypoint_runner , and the /cmd_vel $\rightarrow$ traction path. Real-robot workflows in docs/COMPETITION_WORKFLOWS.md . (implemented — 2D autonomy PoC)
gui	C++/Qt6	Workstation	Operator GUI (video, thermal, dashboard, digital twin, dialogs). Hosts the workstation

Package	Lang	Runs on	Purpose
			bringup.launch.py (GUI + servo + flipper_state + twin) and the arm arm/disarm + dexterity/chassis controls in the dashboard. Video/dashboard/odometry/speech/twin/CV-filters + config dialogs (settings + PPM-calib) implemented.
rescue_vision	Python	Workstation	YOLO hazmat detection on the C920 streams.
arm_description	URDF/ STL	Workstation	Combined robot URDF (arm + chassis + 4 flippers) + chassis/flipper meshes, for the digital twin and the arm's body-collision. (implemented: dicerox_full.urdf ; MoveIt-specific config still deferred)
arm_moveit	config	Workstation	MoveIt 2 config + Servo + launch.
arm_teleop	Python	Workstation	SDLS servo (self- and body-collision), saved-pose RRT planning, keyboard/ joystick teleop, and flipper_state (/encoders/flipper → /joint_states). The one-shot workstation bring-up now lives in gui/bringup.launch.py (the operator console owns it). The arm is jogged from the workstation gamepad (joystick_servo), not the RC.
rescue_bringup	launch/ py	both	Per-host launch + systemd units + two always-on Jetson nodes: robot_manager (start/stop/status for the sensors/mapping/ mapping3d/localization stacks) and map_manager (named map library /robot/maps/* ; drives rescue-localization.service).

(Names are proposals; the legacy equivalents are `esp32_bridge`, `gui`, `jaguar_*` for the arm. Keep names robot-neutral here.)

7. Firmware (ESP32)

PlatformIO project targeting a DOIT ESP32 DevKit V1 (Arduino framework, both cores at 240 MHz). The same firmware runs on both identical PCBs; the only intended per-board build difference is `ROBOCOREA_BOARD_ROLE` in `config.h` (`CHASSIS` by default, or `ARM` for the arm PCB).

7.1 Module structure

Mirror the legacy `lib/` decomposition (clean, worth keeping):

```
firmware/
  ESP/
    platformio.ini
    include/
      config.h          # Board role, pins, CAN IDs, gear ratios, flipper params, protocols
      robot_types.h     # Enums (RobotMode), structs, packed payloads
    lib/
      RC/               # PPM receiver decode (ISR + sync-gap framing)
      Control/          # Mode state machine, fixed RC control mapping, flipper setpoint
      Locomotion/       # Differential mixing + flipper target angle / hold → VESC
      CANInterface/     # CAN HAL (MCP2515 active / TWAI optional); gated by board role
      Comms/            # Binary UART protocol TX/RX
      Gripper/          # End-effector servo (LEDC PWM, GPIO26); ARM role only (§13.4)
      PID/              # Reusable PID (shortest-angle + D low-pass) – spare; flippers lo
      Debug/            # Optional serial debug (mutually aware of ENABLE_COMMS)
    src/
      main.cpp          # setup() + FreeRTOS task creation
  VESC/
    flipper_position.lisp # flipper position loop that runs ON each flipper VESC (§7.3)
```

The legacy firmware also had an `Encoders/` module (PCNT quadrature). **Drop it** for the base: RoboCorea has no separate encoders — position/speed

feedback comes from the VESCs over CAN (§8). Keep PCNT only if a bench test proves VESC feedback is insufficient (see §18).

7.2 FreeRTOS tasks

Role-gated tasks across the two cores. The **arm** board adds two low-priority sensor tasks for the I2C victim-detection sensors; the chassis has none.

Core	Task	Role	Rate	Priority	Purpose
0	<code>commsTask</code>	both	50 Hz	4	UART RX parse + identity + role-owned telemetry
0	<code>canTask</code>	both	200 Hz	4	CAN poll; chassis parses VESCs, arm parses/polls ODrive/ZE300/LKTech
1	<code>controlTask</code>	both	50 Hz	5 (highest)	Chassis: RC/base FSM; arm: relay latest joint command when armed
1	<code>sensorTask</code>	arm	50 Hz	2	LIS3MDL read → <code>MSG_SENSOR_MAG</code>
1	<code>thermalTask</code>	arm	~4 Hz	1 (lowest)	MLX90640 <code>getFrame()</code> (blocks ~250 ms) → quantise → <code>MSG_SENSOR_THERMAL</code>

`controlTask` uses `vTaskDelayUntil` for a strict, drift-free period. The control loop target is **50 Hz**. **The arm always wins:** `sensorTask` /

`thermalTask` sit *below* control on core 1 and never run on core 0, so the heavy MLX float compute only consumes core-1 idle time and is preempted the instant control is ready; they touch only I2C (never the CAN bus). An internal I2C mutex serialises the shared bus between the two — a mag read mid-thermal-frame is skipped, not blocked. Both forward through the existing `Comms` TX mutex.

7.3 Locomotion

- **Traction (2 VESC)**. RC sticks map to forward + turn; the ESP32 mixes them

into left/right targets and sends `SET_RPM` to each traction VESC. The VESC closes the velocity loop internally using its hall sensors. Speed feedback for telemetry is the VESC's reported `eRPM` (status frame). Direction sign and max RPM are config constants.

- **Flippers (4 VESC).** Each flipper is **position-controlled, and the loop runs on the VESC itself** (LispBM — `firmware/VESC/flipper_position.lisp`). The ESP32 is only the setpoint owner; the VESC owns the loop and the feedback:
 - **Stick = rate.** RC stick `[-1, +1]` integrates a per-flipper target angle at `FLIPPER_RATE_DPS * dt`, so the target moves while the stick is deflected and **holds** where it is released (center = hold). Direction sign (`FLIPPER_DIR_*`) is applied here. Optional soft limits, else the target wraps `[0, 360)`.
 - **Transport.** The ESP sends the absolute wrapped target to the flipper VESC in a **custom CAN frame** (`VESC_CMD_FLIPPER_TARGET 0x7E`; *not* `SET_RPM`, which would engage the VESC's own speed loop and fight the lisp's `set-current`). The frame carries an enable flag (false = coast).
 - **On the VESC,** the lisp closes a PD + stiction-feedforward loop with **shortest-path error on a wrapped `[0, 360)` angle** (so it can rotate past 360° continuously), and **reports its measured angle back** in a custom frame (`VESC_CMD_FLIPPER_REPORT 0x7F`).
 - **Bumpless transfer:** on (re)entering flipper control the ESP seeds the target from the VESC-reported angle, so the flippers never jump to a stale setpoint. The reported angle also feeds GUI telemetry.
 - The PD gains and the angle scale (`deg-per-dist`) live **in the lisp**, not `config.h`. *Verifying that scale 1:1 is the key bench task (§18).*

7.4 Sensors (ARM PCB)

Both passive victim-detection sensors live on the **arm PCB's I2C bus** (`Sensors` module + the two tasks in §7.2). They moved off the Jetson I2C, which proved unreliable; the arm ESP32 reads them and forwards them over the UART link, and `esp32_bridge` republishes the same `/sensors/*` topics so the GUI is unchanged.

- **Magnetometer (LIS3MDL).** Read at 50 Hz; XYZ sent as `MSG_SENSOR_MAG` ($3 \times \text{int16 } \mu\text{T}$). 16-gauss range to avoid saturating near the BLDC motors. Only a *heading reference* — the robot's orientation comes from the ZED2.

- **Thermal camera (MLX90640).** 32×24 °C frame at ~4 Hz (8 Hz sub-page refresh; a full frame is half that). The °C frame is **quantised to 8 bits/pixel across each frame's own min/max** (both `int16 ×100 °C` sent in the header, so the Jetson reconstructs true °C) and shipped as `MSG_SENSOR_THERMAL` — ~776 bytes, one UART frame, ~6 KB/s at 4 Hz. The bridge dequantises to a 32×24 `32FC1 Image` on `/sensors/thermal` (+ `_raw`, + `/sensors/thermal_status` with min/max/center/hotspot). The rate + I2C clock are tunable in `config.h` (`THERMAL_REFRESH_RATE`, `SENSOR_I2C_HZ`); drop the rate to give the arm more CPU headroom. *Priority:* the arm comes first — see §7.2.
 - **No IMU on the ESP32.** The BNO055 was removed; orientation/IMU comes from the **ZED2** (visual-inertial). The IMU message type (`0x06`) and its enable bit are kept **reserved-unused** so the protocol numbering stays stable.
 - **Enable mask.** The GUI's `/sensors/enable_mask` (bit0 = mag, bit1 = thermal) is forwarded by the bridge to the arm as `MSG_SENSOR_ENABLE`; `Sensors` gates each sensor so a disabled one never touches the I2C bus. The bridge re-sends the last mask whenever the arm PCB (re)connects.
-

8. CAN bus & motor protocols

Each ESP32 PCB has its **own 500 kbps CAN 2.0 bus** through the same on-board SMD MCP2515 over SPI (`CS=GPI05`, `SCK=GPI018`, `MISO=GPI019`, `MOSI=GPI023`, 8 MHz). The chassis bus carries only the 6 VESCs; the arm bus carries only the mixed-CAN arm. **Every controller must be set to 500 kbps before joining its bus** (ODrive defaults to 250 kbps — change and save it first). The CAN code is backend-agnostic: a compile-time switch in `config.h` selects MCP2515 (active) or the ESP32 TWAI peripheral + SN65HVD230 (future board, placeholder pins). The HAL adds bus-off / fault recovery and a cross-core SPI mutex for the MCP2515 path.

8.1 VESC — traction + flippers (6 controllers)

- Extended 29-bit frames, ID = `(cmd << 8) | vesc_id`.
- **Traction** uses `SET_RPM (cmd 3)`: `int32 BE eRPM` (velocity). (`SET_CURRENT`, `cmd 1`, `int32 BE = A×1000` is kept as a bench fallback.)
- **Flippers** run LispBM on the VESC (§7.3) and use **custom command bytes** in

the same extended-ID scheme — chosen outside the VESC 6.06 `CAN_PACKET_*` set so the firmware ignores them and the lisp's `event-can-eid` handler receives them:

- `0x7E` **target** (ESP → VESC): `[int32 BE millideg][u8 enable]` .
- `0x7F` **report** (VESC → ESP): `[int16 BE deci-deg measured]` .
- Status parsed from the VESC's broadcast frames (big-endian):
 - Status 1 (cmd 9): eRPM, motor current, duty cycle.
 - Status 4 (cmd 16): FET temp, motor temp.
 - Status 5 (cmd 27): tachometer (int32 at data[0..3]) + input voltage. The tachometer is **forwarded** in `MSG_VESC_STATUS` ; the two **traction** VESCs' counts are integrated into **track (wheel) odometry** on the Jetson bridge (`/odom/wheel`). (Flipper *position* still comes from the lisp's `0x7F` report, not this tachometer.) Enable status frames 1/4/5 in VESC Tool for traction/temp/voltage/tachometer telemetry. The ESP gates each status frame by per-command DLC, not `== 8` .
- Forwarded to the PC as `MSG_VESC_STATUS` (0x08), which now carries the tachometer alongside eRPM/current/duty/temps/voltage.
- **CAN IDs** (from `1.ino` / `2.ino` , set in VESC Tool — need **not** be contiguous; the firmware maps id → array index by lookup): traction **L=60, R=50**; flippers **FL=20, FR=10, RL=40, RR=30**. Only the four flipper VESCs run the lisp.

8.2 ODrive — arm J1–J3

- Standard 11-bit frames, COB-ID = `(node_id << 5) | cmd_id` (CANSimple).
- `SET_INPUT_POS` (0x0C): float32 LE turns + int16 LE vel_ff + int16 LE torque_ff.
- `SET_AXIS_STATE` (0x07): uint32 LE state (8 = closed-loop).
- Encoder zero captured at startup via RTR to `GET_ENCODER_ESTIMATES` (0x09) so the boot pose becomes software zero.
- Runtime telemetry via round-robin RTR (~16 Hz/reading): `GET_ENCODER_ESTIMATES` (0x09) pos/vel, `GET_IQ` (0x14), `GET_TEMPERATURE` (0x15), `GET_BUS_VOLTAGE_CURRENT` (0x17). Optional `Get_Error` (0x03).
- Forwarded as `MSG_ODRIVE_STATUS` (0x0A); errors as `MSG_ODRIVE_ERROR` (0x0D).
- Node IDs: **J1=0x10, J2=0x11, J3=0x12**. Gear ratio 48:1, direction `-1` (from legacy `ginkgo_odrive_bridge` config — verify).

8.3 ZE300 — arm J4

- Standard 11-bit frames; **tagged request ID = 0x100 | device_id** , **reply ID = device_id** (different IDs — watch out). Variable DLC (1/5/7 bytes).
- Position in **encoder counts at 16384 counts/output-rev** (driver handles its internal 1:8 gearbox; command in output degrees → counts).
- Startup (blocking, once): **SET_POSITION_MAX_SPEED** (0xB2) then **READ_ABSOLUTE_ANGLES** (0xA3) to capture the zero offset.
- Runtime: **ABSOLUTE_POSITION** (0xC2, fire-and-forget). Disable: **DISABLE_OUTPUT** (0xCF). Low-rate **READ_REALTIME_STATE** (0xA4, ~5 Hz) for telemetry.
- Forwarded as **MSG_ZE300_STATUS** (0x0C). Device ID **J4 = 1**.

8.4 LKTech / MyActuator — arm J5–J6

- Standard 11-bit frames, ID = **0x140 + motor_id** (request and reply share the ID). Command byte = **data[0]** , 8-byte payloads.
- **MOTOR_ON** (0x88) at startup; **READ_MULTI_LOOP_ANGLE** (0x92) once to capture the zero offset (signed 56-bit LE centideg in **data[1..7]**).
- Runtime: **MULTI_LOOP_ANGLE_CONTROL_2** (0xA4): target centidegrees + max speed (dps), fire-and-forget. Telemetry parsed from the 0xA4 ack (temp, lq, speed, angle).
- Forwarded as **MSG_LKTECH_STATUS** (0x0B). Motor IDs **J5=14, J6=15**, gear 10:1 (verify directions on bench).

Arm bring-up reminder: the arm's three controller families are inherited unchanged from the legacy Dicerex arm. All the hard-won protocol details above came from the legacy firmware branch and `dicerox_mixed_motor_config.h` . Cross-check IDs/gear ratios/directions on the bench before any powered motion.

9. ESP32 ⇄ Jetson binary UART protocol

Two USB serial links, **921600 baud** each. Plain binary framing (no micro-ROS).

Both ESP32s periodically send **MSG_BOARD_IDENTITY** , so the Jetson can dynamically

bind whichever `/dev/serial/...` path is the chassis PCB or arm PCB.

Frame: `[0xAA][0x55][TYPE:1][LEN_H:1][LEN_L:1][PAYLOAD:LEN][CRC:1]`

CRC = XOR of `TYPE`, `LEN_H`, `LEN_L`, and every payload byte.

Payload structs are `#pragma pack(1)` and shared conceptually between the firmware (`robot_types.h`) and the bridge (Python `struct`).

9.1 ESP32 → PC

Type	Name	Payload (summary)
0x01	Telemetry	mode, flags, raw PPM[6], speed_l/r (eRPM-derived ×10), flipper angle ×10, uptime
0x02	Thermal	arm PCB: seq, min/max °C ×100, cols/rows, 768 quantised px (~4 Hz)
0x03	Magnetometer	arm PCB: LIS3MDL XYZ, 3× int16 µT (50 Hz)
0x05	Status	mode, flags, sensor mask
0x07	Encoder ext	4 flipper angles ×10 (FL, FR, RL, RR)
0x08	VESC status	per-VESC: id, eRPM, current, duty, FET/motor temp, V_in, tachometer
0x0A	ODrive status	per-joint: idx, pos, vel, Iq, bus V, bus I
0x0B	LKTech status	per-joint (J5/J6) telemetry
0x0C	ZE300 status	J4 telemetry
0x0D	ODrive error	per-node motor_error (optional)
0x0E	Arm lifecycle	arm state, fault, presence mask, operating mode, active joints
0x0F	Board identity	role, protocol version, capability bitmask

0x02 Thermal and **0x03 Magnetometer** are back (same type numbers as legacy), but now sent by the **arm PCB** and republished by `esp32_bridge`

rather than read on the Jetson. Still removed: **0x04 Gas** (sensor dropped), **0x06 IMU** (BNO055 removed — orientation from the ZED2), **0x09 Motor-main** (was `ROBOT_MAIN` -only PWM duties — N/A here); their type numbers stay reserved so the GUI/bridge numbering is stable.

9.2 PC → ESP32

Type	Name	Payload
0x10	Arm joints	6 × int16 (deg ×100) — computed on the workstation
0x11	Sensor enable	arm PCB: 1-byte mask (bit0 mag, bit1 thermal) — from <code>/sensors/enable_mask</code>
0x12	E-stop	(empty) — immediate stop
0x13	E-stop clear	(empty) — resume
0x14	<i>(reserved)</i>	was Keybind — the RC uses a fixed control scheme now (§13.2)
0x15	PPM calibration	6 ch × (min, neutral, max) uint16 + deadband
0x16	Gripper	int16 ×1000 — signed open/close rate (+open / –close) for the arm PCB's servo (§13.4)
0x1A	Traction command	2× int16 normalized L/R track speed ×1000 + u8 enable — autonomy <code>/cmd_vel</code>

(0x17–0x19 are the arm lifecycle commands: init / disarm / mode.)

The bridge translates each of these to/from ROS 2 topics (§12) and routes outbound frames by role: arm joints/lifecycle commands **and the sensor-enable mask** go to the arm PCB; PPM calibration and the traction command go to the chassis PCB; software e-stop is sent to all discovered ESP32 links. The sensor-enable mask (`/sensors/enable_mask`) is forwarded to the arm as `MSG_SENSOR_ENABLE` and re-sent whenever the arm link (re)identifies.

Traction command (0x1A) — the autonomy drive path. The base is normally RC-only; this is the one way ROS can drive the tracks. The Jetson bridge converts

Nav2's `/cmd_vel` into normalized left/right track speeds (using the wheel/track geometry) and sends `MSG_TRACTION_CMD` to the chassis PCB, which feeds them through the **same** VESC eRPM scaling + direction signs the RC path uses (`Locomotion::setTrackSpeeds`). The ESP32 only acts on it while the **RC drive sticks are neutral**, virtual-flip is off, and the command is **fresh** (`EXT_DRIVE_TIMEOUT_MS`) — touching a stick, engaging virtual-flip, losing the RC link, or a stale command all instantly reclaim manual control (that firmware arbitration is *transient* — it re-checks every loop).

On top of that firmware layer the **bridge** owns a *runtime* autonomy gate (an extra software allow/prevent): it only forwards `/cmd_vel` while autonomy is **enabled**, and stops sending on a `/cmd_vel` timeout. The gate's initial value is the `enable_cmd_vel_drive` param (default **false**); the **GUI** toggles it live via `/autonomy/enable`, and the bridge publishes the authoritative state on `/autonomy/state` (latched). Crucially, the bridge makes the handoff **sticky**: when autonomy is engaged and it sees the operator deflect a **drive stick (Ch3/Ch4)** or engage **virtual-flip** (from telemetry), it **latches autonomy off** and releases the tracks — autonomy must then be re-enabled explicitly from the GUI. See §16.

10. Jetson software (relay)

The Jetson runs ROS 2 Humble and acts as the bridge between the robot's hardware and the operator network. Planned nodes:

- **esp32_bridge** (Python). Scans an allowlist of serial device globs (default `/dev/serial/by-id/*,/dev/serial/by-path/*`), opens candidate ESP32 links, and binds them by `MSG_BOARD_IDENTITY` as **chassis** or **arm**. It publishes the same public topics as the old one-port bridge while routing outbound commands to the owning PCB. It also **integrates the two traction VESCs' tachometers into track wheel odometry**, published as `nav_msgs/Odometry` on `/odom/wheel` (with skid-steer covariances; **no TF** — the EKF owns `odom-base_link`). Wheel/track geometry is set via node parameters. **It now also owns the passive sensors**: it decodes the arm PCB's `MSG_SENSOR_THERMAL` / `MSG_SENSOR_MAG` frames, dequantises the thermal frame to

a 32×24 `sensor_msgs/Image` on `/sensors/thermal` (+ `_raw`,
+ `/sensors/thermal_status ThermalStatus`) and publishes `/sensors/mag`
(`sensor_msgs/MagneticField`), and relays `/sensors/enable_mask` (bit0 mag, bit1
thermal) down as

`MSG_SENSOR_ENABLE`. (There is no longer a separate `jetson_sensors` package —
the Jetson I2C path was removed.)

- **C920 SRT streamer** (`gui/scripts/c920_srt_stream.sh`, a GStreamer pipeline,
not a ROS node). One pipeline per C920: the camera's **onboard H.264** (and
the front camera's mic as **Opus**) muxed into MPEG-TS and sent over **SRT** to
the workstation GUI. The Orin Nano has **no NVENC**, so nothing is *encoded* on
the Jetson — the C920 encodes H.264 itself and the Jetson just packetizes
(near-zero CPU). See §11.1 and the gui README.
- **dicerox_mapping** (*implemented*). 2D SLAM front-end + mapper. Projects the
ZED2 VIO into a planar `/filtered_odom` (`zed_planar_odom`), reframes the RPLidar
`/scan` into `base_laser` (`scan_frame_republisher`), and runs `slam_toolbox`
(mapping or localization mode) publishing `map → odom` + `/map`. The ZED2 and
RPLidar **drivers themselves** (`zed_wrapper`, the Slamtec A1 driver) are *external*
sensor bring-up — run them separately, the ZED wrapper with `publish_tf:=false`.
- **rescue_nav** (*implemented — 2D autonomy PoC*). The navigation layer **on top**
of dicerox_mapping: Nav2 (planner + RegulatedPurePursuit controller + costmaps
◦ waypoint follower), the **robot_localization EKF** with an
`adaptive_odom_covariance` node that fuses the bridge's `/odom/wheel` + ZED VIO
+ ZED IMU into a
filtered `odom → base_footprint` (dynamic, condition-aware weighting), a Gazebo
Classic simulation (no hardware), a `waypoint_runner`, and the `/cmd_vel → traction`
drive path (§9.2). The two competition entry points are `real_mapping.launch.py`
(unknown arena) and `real_navigation.launch.py` (known map, AMCL by default) —
see `rescue_nav/docs/COMPETITION_WORKFLOWS.md`. Still 2D only (no elevation map).
- **robot_manager + map_manager** (`rescue_bringup`, Python, always-on).
`robot_manager` starts/stops the perception **stacks** via `systemctl --user`
and reports `/robot/<stack>/status` (sensors, mapping, mapping3d,
localization), so the GUI's **Robot Systems → Perception** tab is just a thin
client (no SSH, no orphaned nodes). `map_manager` owns the **named map library**
under `~/maps/<name>/`: `/robot/maps/{save,list,load,delete}` — 2-D save via
`slam_toolbox serialize_map` + `map_saver_cli`, 3-D via `octomap_node`'s

`/robot/map3d/{save,load}` , and `load` = write the active-map env + (re)start `rescue-localization.service` . See `rescue_bringup/systemd/README.md` .

The Jetson does **not** handle the RF cameras (they go straight to the workstation) and does **not** run the GUI or arm IK.

11. Workstation software (operator)

Ubuntu 22.04 laptop, ROS 2 Humble. The human operator sits here with the RC transmitter. Planned packages:

- **gui** (C++/Qt6). The operator console. Ported almost verbatim from the legacy `gui` package, with the dual-robot support stripped out (Dicerox only). Video, dashboard, odometry, speech, the **digital twin**, the **in-GUI CV filters**, and the **config dialogs** (settings + PPM calibration; no keybind editor — the RC scheme is fixed, §13.2) are implemented. The dashboard also hosts the **arm arm/disarm + dexterity/chassis lifecycle controls** (§16), and the package owns the workstation `bringup.launch.py` . **Detailed design in §11.1.**
- **rescue_vision** (Python). YOLO **hazmat** detection (HazMat label set) on the **C920 streams** (clean digital frames, not the noisy RF driving feed). A trained model + ONNX export exist in the legacy repo (`shared/vision/`); copy it to `gui/assets/vision/best.onnx` for the in-GUI Hazmat filter. The GUI runs inference through **ONNX Runtime** (CUDA EP + CPU fallback), so the export opset is unconstrained.
- **Arm stack** (`arm_description` + `arm_moveit` + `arm_teleop`): the arm is driven from the **workstation gamepad**, not the RC (the RC scheme is drive + flippers only, §13.2).
 - `arm_teleop/joystick_servo` maps gamepad axes to a `geometry_msgs/TwistStamped` on `/servo_node/delta_twist_cmds` .
 - The **SDLS servo** turns that Cartesian twist into joint motion (IK) and publishes the 6 angles on `/joint_states` , which the bridge (on the Jetson) sends down as `MSG_ARM_JOINTS` . (The old RC-stick `rc_servo` path is dropped.)
- **Audio / speech** (*implemented*): the front C920's microphone is encoded as

Opus and muxed into that camera's A/V SRT stream (§11.1). The GUI decodes it natively (GStreamer), plays it on the operator's speakers, and feeds it to **Vosk** for live transcription on the dashboard. No `/audio` ROS topic and no PulseAudio — this replaces the legacy raw-PCM-over-DDS path that was lossy/laggy.

11.1 GUI internals (ported from the legacy `gui`, Dicerox only — video/dashboard/odometry/speech/twin/CV-filters/config-dialogs implemented)

A single C++/Qt6 executable that is also a ROS 2 node. `rclcpp::spin` runs on a **dedicated thread**; ROS callbacks marshal onto the Qt main thread via Qt signals (the legacy `*Updated` signal pattern) — never touch widgets from the ROS thread. Settings persist to `~/.config/robocore_gui/settings.json` via an `AppSettings` singleton.

Window layout (`MainWindow`): a horizontal splitter — **Video** on the left (~2/3), and a right section with the **Digital Twin** on top spanning its width, over the **Odometry** and **Dashboard** panels side by side below. (The odometry readouts used to be a tab beneath the twin; they now sit next to the dashboard so both are visible at once under the twin.)

Components (one class each, as in legacy):

Class	Role
<code>MainWindow</code>	Owns panels; publishes <code>/robot/ppm_calib</code> ; opens <code>SettingsDialog</code> .
<code>CameraHub</code>	Ref-counted frame provider keyed by source URI — one decode per source, shared by widgets, auto-reconnecting. Opens <code>local:N</code> (V4L2) and <code>gst:<pipe></code> (GStreamer/SRT); also hosts externally-registered A/V sources (<code>av:<i></code> , see <code>GstAvStream</code>).
<code>SourceManager</code>	Discovers sources: local webcams (<code>probeLocalCameras</code>), the configured C920 SRT streams (from <code>AppSettings</code>), and thermal ROS topics (<code>probeThermalTopics</code> + a <code>/config</code> subscription). Emits <code>sourcesUpdated</code> .

Class	Role
VideoPanel / VideoWidget	2×2 grid of feeds; click-to-enlarge; each widget runs its own filter pipeline on a worker thread and can select any source (incl. thermal). An enlarged cell supports zoom + pan (on-screen +/-/Fit buttons or Ctrl+scroll to zoom — trackpad pinch only on Wayland, not X11; two-finger scroll / drag to pan), applied as an ROI crop+upscale of the source frame before the filter pipeline (so the CV runs on the zoomed region); resets to fit on collapse.
FilterRegistry / filters	Self-registering CV filters with a thread-safe FilterConfig (atomics). Per-widget instances.
OdometryPanel	Track speeds, track wheel-odometry (x/y/yaw/vx from /odom/wheel) , 4 flipper angles, per-VESC rows (the six VESC IDs) , and arm telemetry (ODrive/LKTech/ZE300).
DashboardPanel	The always-on operator strip: connection LED + heartbeat + uptime + 6S-LiPo pack-voltage readout (color-coded per-cell, from any VESC v_in) , magnetometer readout (+ enable toggle), thermal-acquisition enable toggle , orientation readout from the ZED2 IMU , e-stop button , an AUTO DRIVE toggle (autonomy /cmd_vel allow/prevent, reflecting /autonomy/state ; §16), arm arm/disarm + dexterity/chassis controls , audio toggle, and the ⚙ settings + 📦 Robot Systems toolbar buttons. (The perception/mapping stack controls moved out to SystemsWindow to declutter it.)
SystemsWindow	Top-level "Robot Systems" window (📦 icon next to ⚙), created eagerly+hidden so its clients/subs stay live and a running routine survives the window being closed. A QTabWidget : Perception (PerceptionPanel), Maps (MapsPanel), Arm Program (RoutineProgrammer). Owns stopAllStacks() (called from MainWindow::closeEvent).
PerceptionPanel	The robot_manager stacks — sensors / mapping / mapping3d / localization — each a start/stop + live LED from /robot/<stack>/status , plus Open 2D/3D Map (→

Class	Role
	<code>MapWindow</code>). Moved out of the dashboard. (No I2C stack — thermal/mag are on the arm PCB; their enable toggles stay on the dashboard.)
<code>MapsPanel</code>	The named map library (Maps tab): lists <code>/robot/maps/list</code> names with 2D/3D badges + preview thumbnails, Load (2D/3D/ both → robot-side localization), Delete , Refresh . Map <i>data</i> is robot-owned (<code>map_manager</code>); thumbnails mirror to <code>~/.config/robocorea_gui/map_thumbs/</code> (like arm poses).
<code>RoutineProgrammer</code>	Block-programmed autonomous arm routines (Arm Program tab): chain <i>Go-to-pose</i> / <i>Wait</i> / <i>Gripper open-close</i> / <i>Loop×N</i> / <i>Run-subroutine</i> blocks into named routines (auto-saved to <code>~/.config/robocorea_gui/arm_routines.json</code>). A Qt-thread interpreter (call stack for subroutines, per-frame loop counters) sequences poses on <code>/servo_node/go_to_pose</code> + <code>plan_state==reached</code> , dwells/gripper on a tick timer; Start/ Pause/Stop + selectable top-level loop. Gripper steps publish <code>/gripper</code> (§13.4).
<code>MapWindow</code> / <code>UrdfViewer</code>	Live 2-D <code>/map</code> or 3-D <code>/robot/map3d</code> view (robot at its <code>map→base</code> TF). Toolbar: Set Start Pose (2-D: click-drag on the map → <code>/initialpose</code> , click=X/Y, drag=yaw, roll/pitch 0) and Save Map... (grabs a preview thumbnail, calls <code>/robot/maps/save</code>).
<code>DigitalTwinPanel</code> / <code>UrdfViewer</code>	OpenGL 3-D view of the combined arm + chassis + flipper URDF, posed from <code>/joint_states</code> (arm joints from <code>servo_node</code> , flipper joints from <code>flipper_state</code>).
<code>ArmPosePanel</code>	Saved-pose controls under the twin: a thin client of the <code>servo_node</code> <code>save/go/delete/list</code> services with inline per-pose twin thumbnails. Mirrors the pose-name list + selection to <code>~/.config/robocorea_gui/saved_poses.json</code> and renders to <code>pose_thumbs/</code> so the dropdown is populated at launch before the servo connects (server stays authoritative, §12).

Class	Role
GstAvStream	Native-GStreamer receiver for the front C920's A/V SRT stream: <code>srtsrc ! tsdemux</code> → video appsink (shown via <code>CameraHub</code>) + <code>opusdec ! tee</code> → speakers and an appsink feeding <code>SpeechProcessor</code> . Auto-reconnects.
SpeechProcessor	Vosk transcription, fed 16 kHz PCM by <code>GstAvStream</code> (the C920 A/V stream's Opus track) via <code>pushAudio()</code> .
SettingsDialog / PpmCalibDialog	Robot config: PPM calibration + deadband, thermal colormap/interp, detection-label scale. (No keybind editor — the RC control scheme is fixed, §13.2.)

Cameras (changed from legacy): three source kinds, none through DDS. (1) The **RF driving cams** are local webcams — `CameraHub` opens `/dev/video*` directly (`local:N`). (2) The **two C920s on the Jetson** stream onboard H.264 over **SRT**; the GUI pulls them in directly — the rear cam via OpenCV's GStreamer backend (`gst:<pipe>`), the **front cam's A/V** stream (H.264 + **Opus mic**) via the native `GstAvStream` (`av:<i>`). There is **no** `gst_bridge` and no frame-re-publishing over ROS (the legacy double-hopped video through DDS). (3) The only ROS-borne video is the **thermal** image (`/sensors/thermal`), selectable as a source with a colormap.

Filters / CV: per-widget OpenCV pipelines — **YOLO hazmat** detection (ONNX Runtime, CUDA EP + CPU fallback), QR/barcode (OpenCV or ZBar), and shape detection. Each cell runs its own filter instance with a thread-safe `FilterConfig` exposed as live sliders/toggles under the video. The heavier `rescue_vision` package can also run detection as a separate node; the in-GUI filter is the legacy path.

Topics: publishes `/robot/ppm_calib` (reliable + transient-local), `/robot/estop` (Bool, republished ~10 Hz), and `/sensors/enable_mask` (UInt8: bit0 mag, bit1 thermal; imu bit reserved-unused). Subscribes to the telemetry/sensor/motor/mode/flags topics from §12, plus `/odom/wheel` (track odometry), `/sensors/thermal`, the **ZED2 IMU** topic (orientation readout), and the latched **arm lifecycle** topics

(`/arm/state` , `/arm/operating_mode` , `/arm/can_presence`); the dashboard also **calls** the `/arm/{arm,disarm,mode/dexterity,mode/chassis}` `Trigger` services (§16). The C920 video/audio are **not** ROS topics — they arrive over SRT; the audio-monitor toggle is a local mute, not a published command.

Dicerox-only simplifications vs legacy:

- Drop the `robot_type` (Jaguar/Dicerox) switch in `AppSettings` , `OdometryPanel` , and the settings dialog — the layout is always the 2-traction + 4-flipper, all-VESC drivetrain with the ODrive/ZE300/LKTech arm.
- Drop the **gas** readout/toggle (no MQ2). Keep the magnetometer; the orientation readout now comes from the **ZED2 IMU** (no ESP32 IMU, so no IMU enable toggle).
- Odometry shows the VESC table (6 IDs) + arm telemetry; remove the legacy "main motor / PWM duty" section (that was the Jaguar PWM robot).
- No keybind editor — the RC control scheme is fixed in the firmware (§13.2); the only RC config in the GUI is PPM calibration.

Build deps (Qt6 **incl. OpenGL/OpenGLWidgets**, OpenCV w/ GStreamer, **GStreamer dev + SRT/Opus plugins**, Vosk; and — for the CV filters + digital twin — **ONNX Runtime, Assimp, ZBar, urdf**) are listed in §17 and the gui package README. **PulseAudio is no longer needed** (audio plays via GStreamer `autoaudiosink`). GStreamer is an *optional* build dependency: without its dev headers the GUI still builds, minus the C920 A/V stream + speech audio. The YOLO hazmat model is an optional drop-in (`gui/assets/vision/best.onnx`); absent it, the Hazmat filter shows a "model not loaded" overlay and everything else runs.

12. ROS 2 topic reference

These flow over DDS between the Jetson (`esp32_bridge`) and the workstation (`gui` , arm stack, vision). Names/types follow the legacy contract so the GUI and bridge stay compatible. `esp32_bridge` keeps the public topic names stable even though the data now comes from two identity-bound ESP32 links.

Published by `esp32_bridge` (robot → PC)

Topic	Type	Content
<code>/robot/telemetry</code>	<code>Float32MultiArray</code>	<code>[speed_l_rpm, speed_r_rpm, fl</code>
<code>/robot/mode</code>	<code>String</code>	INIT / STANDBY / NORMAL / ARM
<code>/robot/flags</code>	<code>UInt8</code>	bits: ppm_ok, sensors, can_ok, est
<code>/robot/ppm</code>	<code>Int16MultiArray</code>	raw PPM μ s <code>[ch1..ch6]</code>
<code>/robot/deadband</code>	<code>Float32</code>	normalized deadband (from PPM ca
<code>/robot/status</code>	<code>DiagnosticArray</code>	full diagnostic status
<code>/encoders/tracks</code>	<code>Vector3</code>	x=left_rpm, y=right_rpm (live track s
<code>/encoders/flipper</code>	<code>Float32MultiArray</code>	<code>[fl, fr, rl, rr]</code> degrees
<code>/odom/wheel</code>	<code>nav_msgs/Odometry</code>	track wheel odometry from the tract (no TF)
<code>/autonomy/state</code>	<code>Bool</code> (latched)	actual autonomy-drive state; goes f an RC override (§9.2/§16)
<code>/motors/vesc_status</code>	<code>Float32MultiArray</code>	per-VESC telemetry (incl. tachomet
<code>/motors/odrive_status</code>	<code>Float32MultiArray</code>	per-arm-joint telemetry
<code>/sensors/thermal</code>	<code>Image</code>	32×24 float32 °C — decoded from <code>MSG_SENSOR_THERMAL</code>
<code>/sensors/thermal_raw</code>	<code>Image</code>	same frame (no temporal filter appli
<code>/sensors/thermal_status</code>	<code>mlx90640_msgs/ThermalStatus</code>	seq, sensor_ok, min/max/center, ho
<code>/sensors/mag</code>	<code>MagneticField</code>	LIS3MDL XYZ (Tesla) — from the a <code>MSG_SENSOR_MAG</code>

Subscribed by `esp32_bridge` (PC → robot)

Topic	Type	Content
<code>/robot/estop</code>	<code>Bool</code>	true = e-stop, false = clear
<code>/joint_states</code>	<code>sensor_msgs/JointState</code>	arm joint positions (rad), forwarded to the arm ESP32
<code>/robot/ppm_calib</code>	<code>UInt16MultiArray</code>	6ch × (min, neutral, max) + deadband; also cached for the autonomy override check
<code>/gripper</code>	<code>Float32</code>	gripper open/close rate (+open / –close), routed to the arm PCB as <code>MSG_GRIPPER</code> (§13.4)
<code>/cmd_vel</code>	<code>geometry_msgs/Twist</code>	autonomy drive (Nav2); forwarded as <code>MSG_TRACTION_CMD</code> only while autonomy is enabled (§9.2)
<code>/autonomy/enable</code>	<code>Bool</code>	operator request to allow/prevent <code>/cmd_vel</code> driving the tracks (from the GUI; volatile)
<code>/sensors/enable_mask</code>	<code>UInt8</code> (latched)	bits: mag (0), thermal (1); relayed to the arm PCB as <code>MSG_SENSOR_ENABLE</code>

Published by `gui`

`/robot/ppm_calib` (reliable + transient-local), `/robot/estop` (republished ~10 Hz), `/sensors/enable_mask`, `/autonomy/enable` (`Bool`, the autonomy allow/prevent toggle in the dashboard), `/gripper` (`Float32` rate, from the `RoutineProgrammer` gripper blocks; the gamepad also publishes it, §13.4), and `/initialpose` (`PoseWithCovarianceStamped`, map frame — from a click-drag on the 2-D `MapWindow`, consumed by AMCL/slam_toolbox localization). The dashboard also **subscribes** to `/autonomy/state` (latched) to drive the toggle's

checked state, so an RC override that the bridge latches off is reflected in the button. (No `/robot/keybind` — the RC control scheme is fixed, see §13.2. No `/audio` topic — the C920 audio rides SRT and is decoded in the GUI.)

Robot Systems window services (the GUI is a client; servers run on the Jetson):
`map_manager` exposes `/robot/maps/{save,list,load,delete}` (`rescue_interfaces` `SaveMap` / `ListMaps` / `LoadMap` / `DeleteMap`) — named maps under `~/maps/<name>/` ; 2-D save = `slam_toolbox` `serialize_map` + `nav2_map_server` `map_saver_cli` , 3-D save/load forwarded to `octomap_node` 's `/robot/map3d/{save,load}` , 2-D load restarts `rescue-localization.service` on the chosen map. `robot_manager` exposes the `/robot/<stack>/{start,stop,restart}` + `/robot/<stack>/status` set for `sensors,mapping,mapping3d,localization` (`PerceptionPanel`).

Arm stack

`joystick_servo` (workstation gamepad) → `/servo_node/delta_twist_cmds` (`TwistStamped`); the SDLS servo integrates that into `/joint_states` (which the bridge forwards as `MSG_ARM_JOINTS`).

Saved poses / go-to-pose (served by `servo_node` , used by the GUI digital-twin panel and CLI/RViz; defs in the `rescue_interfaces` package):

Interface	Type	Content
<code>~/save_pose</code>	<code>rescue_interfaces/srv/SavePose</code>	snapshot current EE pose under a name
<code>~/go_to_pose</code>	<code>rescue_interfaces/srv/GoToPose</code>	IK + RRT plan a collision-free path to a save execute it
<code>~/delete_pose</code> / <code>~/list_poses</code>	<code>rescue_interfaces/srv/*</code>	manage the pose library
<code>~/ee_pose</code>	<code>geometry_msgs/PoseStamped</code>	live end-effector pose (~20 Hz)
<code>~/plan_state</code>	<code>std_msgs/String</code> (latched)	<code>idle</code> / <code>planning</code> / <code>moving</code> k/n / <code>reached</code> /

The pose library persists **server-side** — the `servo_node` is the source of truth, writing `~/config/robocorea_arm/poses.yaml` (name → EE pose + joint

snapshot) on every save/delete and reloading it at startup, so poses survive servo restarts, rebuilds and reboots. The **GUI** keeps a thin local mirror in `~/.config/robocore_gui/ : saved_poses.json` (the pose-name list + the selected name, so the dropdown is populated instantly at launch before the servo is reachable) and `pose_thumbs/<name>.png` (the per-pose twin render shown inline in the dropdown — the servo does not store these). On startup the panel shows the cached names, then calls `list_poses` to reconcile with the servo (authoritative); the cache is refreshed on every save/delete/list. Planning runs on a worker thread; a non-zero stick/twist, a fault, or `pause_servo` aborts an in-progress move (operator override). See `arm_teleop/README.md`.

13. Operating modes, RC link & control loops

13.1 Mode state machine (on the ESP32)

`RobotMode` : `INIT` → `STANDBY` → `NORMAL` , plus `ESTOP` . (The legacy `FLIPPER` and `ARM` modes are gone — see §13.2 — but their enum values are kept reserved for protocol stability.)

Mode	Behavior
INIT	Hardware init at boot.
STANDBY	Idle, waiting for a valid RC link.
NORMAL	RC drives the tracks and flippers simultaneously (the fixed scheme below). Workstation arm-joint commands are relayed concurrently.
ESTOP	All outputs neutralized; cleared by the PC (<code>MSG_ESTOP_CLEAR</code>) or by returning the Ch6 lever to centre.

13.2 RC control scheme (fixed)

There is **no operator-editable keybind table** anymore (the old `ChannelFunction`

table + `MSG_KEYBIND` were removed). The RC uses one **fixed** mapping, hardcoded in the firmware (`firmware/ESP/lib/Control/Control.cpp` , channel roles in `config.h`). Drive + flippers are always live together; the **arm is driven from the workstation gamepad** (`joystick_servo`), not the RC.

Channel	Type	Role
Ch3	stick	traction forward / back
Ch4	stick	traction turn
Ch2	stick	flipper rate — drives whichever flipper(s) Ch1+Ch5 select (stick = rate, holds where released; loop closed on the VESC)
Ch1	stick → 3-way	flipper L/R selector : below –deadband = left only, within deadband = both , above +deadband = right only
Ch5	2-state lever	flipper pair : min = front (FL/FR), max = rear (RL/RR)
Ch6	3-position lever	down = E-STOP , centre = normal, up = virtual-flip

So at any time Ch1+Ch5 pick one flipper (or a left/right pair) and Ch2 moves it; flip Ch5 to switch to the other end, nudge Ch1 to isolate a side.

Virtual flip (Ch6 up) — "drive from the other end". The robot is symmetric, so to back out of a dead-end hallway without turning around, Ch6-up does a full 180° remap of the control frame: traction forward is negated (steering is left as-is — negating forward already produces correct reverse-drive turning, so inverting turn too would double-flip it), the Ch5 front/rear pair selection swaps, and the Ch1 left/right flipper selection mirrors (operator-left = robot-right). Each piece is a bench-tunable sign (`VFLIP_*` in `config.h`). The active state is reported to the GUI in the telemetry flags (bit4), shown as **"REVERSE"** in the odometry panel.

E-stop (Ch6 down) is level-based: hold the lever down to stop, return it to centre to resume (unless a software e-stop is also active). See §16.

13.4 Gripper (end-effector servo, arm PCB)

The gripper is the one actuator driven directly by **PWM** — a single JX CLS-12V7346 coreless servo on the **arm PCB's** `GPI026` (LEDC), one servo moving both jaws via a linkage. It is **not** on the CAN bus and **not** part of the arm IK. Only the firmware built for the `ARM` board role brings it up.

Command path (workstation-in-the-loop, like the arm joints):

```
gamepad RT/LT → joystick_servo (rate = RT - LT, +open/-close)
  → /gripper (Float32) → esp32_bridge → MSG_GRIPPER (0x16, int16 ×1000)
  → _send_to_role(ARM) → (serial) → arm ESP32 Gripper module
  → integrate rate → clamp [GRIPPER_CLOSED_DEG, GRIPPER_OPEN_DEG] → LEDC servo
```

Holding RT opens, holding LT closes; the target **steps until a limit**, then holds. The firmware integrates at `GRIPPER_RATE_DPS` (config.h) every control tick. A stale command (`GRIPPER_CMD_TIMEOUT_MS`) stops stepping; an E-stop freezes the servo at its current angle. Power the servo from the **11.1–15 V** rail (not the ESP32), common-ground the signal. Bench sketch: `firmware/servo_test` .

This reverses the legacy `MSG_GRIPPER` direction: it was ESP→PC (an RC value the PC ignored); it is now **PC→ESP**, routed to the arm board (matching the §9.2 / §12 contract).

13.5 The two control loops

Base (closed entirely on the robot, low latency):

```
operator stick → RC TX → 2.4 GHz → FS-IA6B → PPM → ESP32
  → traction: mix → SET_RPM → VESC → motor      (VESC closes the velocity loop)
  → flippers: integrate → target angle → custom CAN frame → VESC LispBM → motor
                                     ↑ VESC closes the position loop + reports the an
```

Arm (closed through the network, operator-in-the-loop): the arm is **not** on the RC anymore — it is driven by the workstation gamepad:

```
operator gamepad → joystick_servo → TwistStamped → SDLS servo (IK) → /joint_states
→ (network) → Jetson esp32_bridge → MSG_ARM_JOINTS → (serial) → ESP32
→ arm CAN relay → ODrive/ZE300/LKTech → joints    (gated by the arm lifecycle)
```

The ESP relays these joint commands continuously (concurrent with RC drive/flippers); the arm only moves once it has been **armed** (§16). If the network link drops, the arm freezes (no new joint commands) but the base still responds to RC. If the **RC** link drops, the tracks stop and the **flippers hold their angle** (they are never driven home).

14. Coordinate frames / TF

- `base_link` — robot body origin.
- Arm chain: `base_link` → `Link1` ... `Link6` (end effector), from the arm URDF. Movelt Servo's default twist frame is the end-effector link (`Link6` in legacy) for local jogging, or `base_link` for world-frame jogging.
- No IMU frame on the ESP32 side. The robot's IMU is the **ZED2's**, on `zed_camera_link` (mounting transform to `base_link` set in the URDF).
- (*Nav, implemented*) The nav stack's **standardized TF contract** (set up by `rescue_nav/launch/sensor_frontend.launch.py`) is:
`map` → `odom` → `base_footprint` → `base_link` → `base_laser` .
 - `map` → `odom` — `slam_toolbox` (mapping) or AMCL/ `slam_toolbox` (localization).
 - `odom` → `base_footprint` — the **robot_localization EKF** (fuses `/odom/wheel` ▪ ZED VIO + ZED IMU). The wheel-odom publisher deliberately emits **no** TF (the EKF owns it); the ZED wrapper must run with `publish_tf:=false` .
 - `base_footprint` → `base_link` — a **static identity** transform (the planar rescue base treats them coincident; refine `z` on hardware). This is what lets the robot/arm URDF (rooted at `base_link`) attach under the nav frames.
 - `base_link` → `base_laser` — static RPLidar extrinsics (A1 mounted reversed → `yaw =  $\pi$` by default).

Note: both real-robot stacks now share this one front-end —
`rescue-mapping.service` (`real_mapping.launch.py`) and
`rescue-localization.service` (`real_navigation.launch.py`) each include

```
sensor_frontend.launch.py , so mapping and localization produce the same canonical tree (no base_link -vs- not divergence between them). The dicerox_mapping/mapping.launch.py file remains only as a standalone dev tool with the shorter odom → base_footprint → base_laser tree — not used by any production stack. The bridge's /odom/wheel is stamped child_frame=base_link (coincident with base_footprint ); the EKF's adaptive_odom_covariance restamps it to base_footprint before fusion.
```

The digital twin in the GUI consumes the **combined** robot URDF (`arm_description/dicerox_full.urdf` — arm + chassis + flippers) on `/robot_description` and renders the live pose from `/joint_states` : the arm joints from `servo_node` , the flipper joints from `arm_teleop/flipper_state` (which maps `/encoders/flipper` → the URDF's `Flipper1J..Flipper4J`).

15. Networking

- The ROS 2 middleware is **Zenoh** (`rmw_zenoh`), not the default Fast-DDS. Each host runs a per-host **Zenoh router** (a system service) configured for **unicast TCP** to the peer (multicast disabled, gossip + auto-retry) — reliable on congested arena Wi-Fi. ROS nodes are Zenoh peers connecting to their local router (`localhost:7447`); the two routers federate discovery over one TCP link. Static IPs: **Jetson** `192.168.50.10` , **laptop** `192.168.50.11` . Set up once per host with `scripts/setup-zenoh.sh` (installs the router, makes `rmw_zenoh_cpp` the default RMW, pins `ROS_DOMAIN_ID`).
- Jetson and workstation share one **ROS_DOMAIN_ID** (default **20**) over Wi-Fi/Ethernet. No micro-ROS; the only serial link is ESP32↔Jetson.
- Large/continuous data (thermal image) uses appropriate QoS; config/PPM calibration uses **reliable + transient-local** so a late-joining GUI or bridge still receives the latest value.
- **Bulk media does not ride ROS.** The RF *driving* cameras are local webcams at the workstation; the C920 *inspection* cameras stream H.264 (+ Opus audio) over **SRT** (selective retransmission within a fixed latency budget — tuned for the lossy arena Wi-Fi). Only the 32×24 thermal image rides ROS. The Jetson is the SRT listener; the GUI is the caller and reconnects on drop.

16. Safety & e-stop

- **RC e-stop (Ch6 down):** Ch6 is a 3-position lever — its **down** third triggers `MSG_ESTOP` directly on the chassis ESP32; centre = normal, up = virtual-flip (§13.2). Level-based: returning the lever to centre resumes the chassis (unless a software e-stop is also latched). **Safety tradeoff for this revision:** the arm PCB does not have a direct RC/e-stop wire. The Jetson bridge mirrors chassis e-stop transitions to the arm ESP32 over the second USB serial link (`MSG_ESTOP` / `MSG_ESTOP_CLEAR`). The mirror is **only authoritative while the chassis link is present**: if the chassis serial link drops, the bridge clears the mirror and releases the arm (sends `MSG_ESTOP_CLEAR` , unless a software e-stop is independently latched), so the arm can be **armed with only the arm PCB connected** (bench/standalone-arm work). While the chassis *is* connected its e-stop still holds the arm as before, and the GUI **software e-stop** stops the arm regardless of the chassis.
- **Software e-stop:** the GUI publishes `/robot/estop` (republished ~10 Hz); the bridge broadcasts `MSG_ESTOP` / `MSG_ESTOP_CLEAR` to all discovered ESP32 links. ESTOP neutralizes the **base** outputs (tracks stop, flippers hold) and **holds** the arm; recovery requires an explicit clear.
- **Autonomy drive gate + handoff (three layers).** Nav2's `/cmd_vel` can only move the tracks through `MSG_TRACTION_CMD` , and it is guarded at three levels:
 - i. **Firmware arbitration (transient, always on):** the chassis ESP32 obeys the external command *only* while the RC drive sticks are neutral, virtual-flip is off, and the command is fresh — re-checked every control loop (§9.2).
 - ii. **Bridge runtime gate (software allow/prevent):** the bridge forwards `/cmd_vel` only while autonomy is **enabled**. The operator owns this from the GUI dashboard **"AUTO DRIVE"** toggle (`/autonomy/enable`); the bridge reports the real state on `/autonomy/state` (latched). Default is **off** (`enable_cmd_vel_drive`).
 - iii. **Latched handoff to teleop:** when autonomy is engaged and the bridge sees the operator deflect a **drive stick** or engage **virtual-flip** (from telemetry), it **latches autonomy off** and releases the tracks. Re-engaging autonomy is a deliberate GUI action — a stick nudge does not silently resume it. (E-stop is separate: Ch6-down or the software e-stop still stops everything regardless.)

- **Arm e-stop = HOLD, not limp (`ARM_ESTOP_HOLD = 1` , default):** an e-stop on an *armed* arm freezes every joint at its last commanded pose with the motors **still energized**, so the (heavy, gravity-loaded) arm stays put instead of dropping. The arm control tick re-issues the hold every loop while in ESTOP, new joint commands are ignored, and clearing the e-stop resumes motion **in place** with no re-arm (the firmware ramp eases from the hold pose to the live target). In **Chassis mode** the wrist (J5/J6) is already torque-off, so the hold leaves it off. With `ARM_ESTOP_HOLD = 0` (legacy) e-stop instead de-energizes every joint and drops to UNINIT (the arm droops and needs a fresh **arm**). A latched **CAN fault** always de-energizes regardless of this flag.
- **RC failsafe:** if no valid PPM frame arrives within the timeout (`PPM_TIMEOUT_MS`), the tracks stop and the flippers **hold** their angle. An e-stop (hardware **or** the GUI software stop) also **holds** the flippers in place by default (`FLIPPER_ESTOP_HOLD = 1`): both paths run `Locomotion::estopOutputs()` , which keeps the VESC position loop closed at the last commanded target. Note the active fake-RPM lisp path (`FLIPPER_USE_LEGACY_RPM_LISP`) has **no coast bit** — setting `FLIPPER_ESTOP_HOLD = 0` there drives the flippers to *angle 0* instead of free-wheeling, so leave it at `1` to hold.
- **Arm lifecycle (passive boot):** the arm boots **disarmed** (`ARM_PASSIVE_BOOT`) — the CAN HAL comes up but no motor is enabled, no zero is captured, and joint commands are ignored until the operator **arms** it. The Jetson bridge exposes this as `std_srvs/Trigger` services `/arm/arm` , `/arm/disarm` , `/arm/mode/dexterity` , `/arm/mode/chassis` , and latches the state on `/arm/state` (UNINIT/INITIALIZING/READY/FAULT), `/arm/fault` , `/arm/can_presence` (per-joint init bitmask), `/arm/operating_mode` and `/arm/joint_active_mask` . The GUI dashboard surfaces all of this (state LED, CAN-presence dots, Arm/Disarm + a Dexterity↔Chassis toggle) and offers a one-shot **"arm now?"** prompt at startup when it finds the arm disarmed. **Chassis mode** torque-offs the wrist (J5/J6) so they don't overheat while the arm is parked. A latched CAN fault disarms the arm and rejects motion until a fresh arm; the SDLS servo holds (publishes nothing) while faulted/disarmed.
- **Bring-up discipline:** confirm CAN bitrate (500 kbps everywhere), VESC/arm IDs, gear ratios, and direction signs **before** powered motion. Start motor tests with small commands. Re-check ODrive/LKTech/ZE300 zero offsets after

power cycles or manual repositioning.

17. Build & run

Firmware (ESP32)

```
cd firmware/ESP
pio run          # build
pio run -t upload # flash
pio device monitor # serial monitor
```

Edit `include/config.h` for pins, CAN IDs, gear ratios, flipper params, and the single per-board role macro. Leave `ROBOCOREA_BOARD_ROLE` at `ROBOCOREA_BOARD_ROLE_CHASSIS` for the chassis PCB; set it to `ROBOCOREA_BOARD_ROLE_ARM` for the arm PCB before building/flashing that board. The flipper VESCs also need `firmware/VESC/flipper_position.lisp` flashed via VESC Tool (Scripting → LispBM, "Run at startup") — see `firmware/VESC/README.md`.

ROS 2 workspace (Jetson and workstation)

```
cd software/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash
```

One-command bring-up (the easy path): `./jetson.sh` on the robot and `./laptop.sh` on the workstation (repo root) do everything below as managed services. The full operator runbook — map-while-teleop, save, localize + navigate, fresh-reboot bring-up, parameter overrides — is `OPERATIONS.md`. The manual commands here are what those scripts automate.

On the Jetson launch the relay + the C920 video/audio streamer. The thermal camera + magnetometer come up automatically with the bridge (they're read on the

arm PCB and republished here — no separate sensor node):

```
ros2 launch esp32_bridge esp32_bridge.launch.py
# Optional explicit bench override:
# ros2 launch esp32_bridge esp32_bridge.launch.py serial_port:=/dev/serial/by-id/usb-...
./software/ros2_ws/src/gui/scripts/c920_srt_stream.sh # C920 H.264 (+0pus) → SRT; edit c
# Autonomy (2D PoC): build a map, then navigate a known map. ZED + RPLidar drivers
# are external sensor bring-up (run the ZED wrapper with publish_tf:=false).
#   ros2 launch rescue_nav real_mapping.launch.py          # unknown arena → build map
#   ros2 launch rescue_nav save_competition_map.launch.py map_name:=$HOME/maps/track
#   ros2 launch rescue_nav real_navigation.launch.py       # known map (AMCL default)
# See rescue_nav/docs/COMPETITION_WORKFLOWS.md. Enable the drive path on the bridge
# with enable_cmd_vel_drive:=true (or the GUI "AUTO DRIVE" toggle, §16).
```

On the workstation launch the operator stack in one shot (GUI + SDLS servo + flipper_state + digital twin; add `joystick:=true` for gamepad arm teleop):

```
ros2 launch gui bringup.launch.py          # GUI + twin + servo
ros2 launch gui bringup.launch.py joystick:=true # + gamepad arm teleop
```

Per-host one-command bring-up lives at the repo root: `jetson.sh` (robot: installs the systemd units, starts `esp32-bridge` + `robot-manager` + `map-manager` + `c920-stream`, leaves perception on-demand for the GUI) and `laptop.sh` (workstation: `gui bringup.launch.py`). Both reconcile the Zenoh router + domain and take overrides (`--serial`, `--no-ekf`, `--domain`, ...). `diagnose.sh` is the read-only hardware diagnostics dispatcher (PPM, CAN presence for the 6 VESCs + 6 arm joints, arm-PCB I2C sensors, comms/link health — `esp32_bridge diag_*` nodes). The robot-side perception/maps systemd layer is in `rescue_bringup` (`robot_manager` + `map_manager` + `systemd/`); see its README and `OPERATIONS.md`.

Workstation extra deps: Qt6 (incl. `libqt6opengl6-dev`), OpenCV w/ GStreamer, **GStreamer dev + runtime plugins**

(`libgstreamer1.0-dev libgstreamer-plugins-base1.0-dev gstreamer1.0-plugins-{base,good,bad}` — the SRT element is in `plugins-bad` and pulls `libsrt`), and

Vosk (speech). For the CV filters + digital twin: ONNX Runtime (1.20.x),

Assimp (`libassimp-dev`), ZBar (`libzbar-dev`), and `ros-humble-urdf` . **No**

PulseAudio (audio plays via GStreamer). Details in the gui package README.

18. Open decisions & TODO

Things that are **not yet pinned down** and must be resolved on real hardware:

1. **Flipper position loop.** *Decided:* the loop runs **on the VESC** (LispBM, `firmware/VESC/flipper_position.lisp`) — PD + stiction feedforward, shortest-path on a wrapped `[0,360)` angle. The ESP integrates the stick into a target angle and exchanges target/measured-angle over custom CAN frames (§8.1). **Bench-verify** the lisp's `deg-per-dist` scale tracks physical rotation 1:1, confirm the VESC 6.06 LispBM API (`event-can-eid` , `can-send-eid` , `conf-get 'controller-id` , ...), and that `0x7E` / `0x7F` don't collide with a `CAN_PACKET_*` .
2. **VESC CAN IDs / baud.** Set from `1.ino` : traction **60/50**, flippers **20/10/40/30** (need not be contiguous). Confirm each in VESC Tool; set every VESC's CAN baud to **500 kbps** (or move the whole bus to 1 Mbps via `CAN_BITRATE_BPS`). Flash the lisp on the four flipper VESCs only.
3. **VESC command mode.** Traction = `SET_RPM` (velocity); tune `TRACTION_ERPM_MAX` . Flippers = lisp `set-current` (position loop). VESC `SET_CURRENT` (cmd 1) kept as a bench fallback.
4. **PCB pin map.** Reconcile every pin in `config.h` against the custom PCB (I2C, CAN SPI, PPM input) — current values come from `1.ino` / `2.ino` .
5. **Flipper tuning.** PD gains live in `flipper_position.lisp` (`kp/kd/fric/i-max`); the stick feel is `FLIPPER_RATE_DPS` in `config.h` .
6. **Arm IDs / gears / directions.** ODrive (0x10–0x12, 48:1), ZE300 (id 1, 16384 cpr), LKTech (14/15, 10:1) inherited from legacy — re-verify each.
7. **Flipper angle limits.** Default is free 360° spin (wrapped). To range-limit, set `FLIPPER_SOFT_LIMIT_ENABLE` + `FLIPPER_ANGLE_MIN/MAX` (switches the ESP target from wrapped to clamped).
8. **Autonomy (2D PoC built).** *Decided:* 2D. Mapping is built — `dicerox_mapping` does 2D SLAM with `slam_toolbox` , taking **ZED2 odometry** (filtered to a planar `odom` → `base_footprint` by `zed_planar_odom`) + the RPLidar (`/scan` reframed to `base_laser` by `scan_frame_republisher`), and supports map

save + `slam_toolbox` localization. TF tree:

`map → odom → base_footprint → base_laser` . `rescue_nav` builds the **Nav2** layer on top (planner +

RegulatedPurePursuit controller + costmaps + waypoint follower), a **Gazebo**

Classic simulation of the tracked base, a **waypoint_runner** (drive to the end

of a track and back), and the `/cmd_vel → traction` drive path (firmware

`MSG_TRACTION_CMD 0x1A` + bridge subscriber, §9.2). The robot-side 3D mapping

add-on is `rescue_mapping3d` : it consumes the ZED registered point cloud

locally on the Jetson, inserts a downsampled cloud into OctoMap, and publishes

only the compact binary octree on `/robot/map3d` for the GUI's 3D voxel view.

The raw ZED point cloud must remain robot-local so it cannot saturate the DDS Wi-Fi link.

9. **Robot/workspace naming.** This doc uses robot-neutral package names (`rescue_*`); the legacy used `jaguar_*` / `dicerox` . Pick final names when creating packages.
10. **Audio/speech.** *Decided/implemented:* the front C920's mic → **Opus**, muxed into that camera's A/V **SRT** stream; the GUI decodes it (GStreamer), plays it, and runs **Vosk** transcription. Replaces the legacy raw-PCM-over-DDS `/audio` path (lossy + laggy). Drop a Vosk model into `gui/assets/audio/` to enable transcription.
11. **Video/audio SRT link.** *Decided:* C920 onboard H.264 (+ front-cam Opus) → SRT → GUI; RF cams stay analog for driving. Set the Jetson IP + per-camera SRT ports in the GUI settings and `c920_srt_stream.sh` , and bench-test bitrate / keyframe interval / SRT latency under a degraded link. CV (YOLO hazmat) runs on the clean C920 streams, not the RF driving cams.
12. **Odometry fusion (EKF built; map-frame EKF deferred).** *Decided + built:* a `robot_localization` **EKF** in `rescue_nav` fuses the bridge's `/odom/wheel` (track tachometers) with the **ZED2** IMU + VIO into a filtered `odom → base_footprint` . An `adaptive_odom_covariance` node sits in front of the EKF and modulates each source's covariance from live signals (slip, tilt, turn rate, ZED pose-cov) so the EKF rebalances itself — wheel `vx` trusted in straight lines, ZED IMU gyro trusted for yaw, ZED VIO for lateral. It runs as a single `odom` -frame EKF (`two_d_mode: true`); the ZED driver must run with `publish_tf: false` so it doesn't fight the EKF. **Still deferred:** the second, `map` -frame (global) EKF — add when a global heading/position reference is wired.

Bench items: measure `wheel_circumference_m` and `track_width_m`; set `traction_dir_*` and the bridge `gear_ratio` to match `config.h` (`TRACTION_GEAR_RATIO=23.333`); confirm the VESC tachometer's steps-per-erev; and tune the skid-steer covariances (forward `vx` trusted, yaw distrusted — heading comes from the ZED2 IMU).

13. **Passive sensors on the arm PCB (built; bench-verify).** *Decided + built:* the **MLX90640 thermal camera + LIS3MDL magnetometer** moved off the (flaky) Jetson I2C onto the **arm PCB** I2C bus (`SDA=21`, `SCL=22`), read by two low-priority arm-ESP32 tasks and forwarded over UART; `esp32_bridge` republishes `/sensors/thermal` (+ `_raw`, `_status`) + `/sensors/mag` and relays `/sensors/enable_mask` down (§7.4, §9). Thermal is 8-bit-per-pixel quantised across each frame's min/max at **~4 Hz** to stay tiny on the UART and cheap on the arm CPU. **The arm is the priority:** sensor tasks sit below control/CAN and never touch the CAN bus. **Bench items:** confirm both I2C addresses (MLX `0x33`, LIS3MDL `0x1C` / `0x1E`) and that the bus is clean alongside the arm wiring; watch that `thermalTask` doesn't measurably perturb the arm CAN relay under load (if it does, drop `THERMAL_REFRESH_RATE` or, in the worst case, move the sensors to the chassis PCB); raise `SENSOR_I2C_HZ` toward 1 MHz only if you push the refresh up; and sanity-check that the quantised °C reconstructs faithfully in the GUI thermal overlay.
-

19. Glossary

- **PPM** — Pulse-Position Modulation; the FlySky receiver multiplexes 6 RC channels onto one signal wire decoded by the ESP32.
- **TWAI** — "Two-Wire Automotive Interface", Espressif's name for the ESP32's built-in CAN 2.0 controller (with an SN65HVD230 transceiver). Kept as an **optional** CAN backend; this board's active transceiver is an on-board MCP2515.
- **VESC** — open-source BLDC motor controller; takes velocity/current/position commands over CAN and broadcasts status frames.
- **CANSimple** — ODrive's CAN protocol (`C0B-ID = (node_id<<5) | cmd_id`).
- **Movelt Servo** — real-time Cartesian/joint jogging for the arm (twist → IK).
- **Differential / tank drive** — steering by varying left vs. right track speed.

- **Flipper** — articulated track-arm that rotates to climb obstacles/stairs.
- **eRPM** — electrical RPM (mechanical RPM $\times$ pole pairs), the VESC's native speed unit.
- **Relay (Jetson)** — the Jetson's role: bridge serial $\leftrightarrow$ ROS and pass data between robot and workstation without owning the GUI or arm IK.
- **Dicerox** — this robot's predecessor in the legacy TMR2026_Rescue repo; "Jaguar" there is an unrelated platform to ignore.

